# D1 Telemetry Dataset Manual

2026-05-04

# D1 Telemetry Dataset

The Cloudflare D1 SQLite database that holds raw scan records. Feeds the annual State of Fintech Security report and the peer-comparison percentiles.

## Identifiers

| | VALUE |
|---|---|
| Database name | `preston-check-telemetry` |
| Database ID | `e206e1e4-1c78-4a5e-a983-bc47104d1b3c` |
| Region | ENAM (eastern North America) |
| Binding name (in code) | `DB` |

Database ID is a public identifier, not a credential.

## Schema

One table, two indexes. Live schema in `workers/telemetry/schema.sql`:

```
CREATE TABLE scans (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  repo_hash   TEXT NOT NULL,    -- SHA-256 hex of remote origin URL
  version     TEXT NOT NULL,    -- Preston-Check release version
  tier        TEXT NOT NULL,    -- free | pro | enterprise
  lang        TEXT NOT NULL,    -- detected primary language
  pass        INTEGER NOT NULL,
  fail        INTEGER NOT NULL,
  warn        INTEGER NOT NULL,
  skip        INTEGER NOT NULL,
```

```
  total       INTEGER NOT NULL,
  score       INTEGER NOT NULL, -- pass*100/total, clamped 0–100
  timestamp   TEXT NOT NULL,    -- ISO 8601 UTC from the client
  inserted_at INTEGER DEFAULT (strftime('%s','now'))
);

CREATE INDEX idx_scans_repo_hash ON scans(repo_hash);
```

`repo_hash` is intentionally not a unique key — multiple scans from the same repo over time produce multiple rows, which is the trend data the SaaS uses for the score-over-time chart.

# Retention

| | POLICY |
|---|---|
| Raw rows in `scans` | 90 days, then deleted by a scheduled cleanup |
| Aggregate counters in KV | Indefinite |
| Score histograms in KV | Indefinite |

The 90-day TTL is privacy-driven: the dataset is useful for the annual report and trend analysis, but not for indefinite tracking of any single repo. After 90 days the row is gone.

The aggregate counters (KV) lose nothing on row deletion because they were already incremented at write time.

# Querying

Operator-only — requires the Cloudflare API token:

```
export CLOUDFLARE_API_TOKEN="..."   # from repo secrets, or operator's local
export CLOUDFLARE_ACCOUNT_ID="..."

# Total scan count
wrangler d1 execute preston-check-telemetry --remote \
  --command "SELECT count(*) AS n FROM scans"

# Daily volume — last 7 days
wrangler d1 execute preston-check-telemetry --remote \
  --command "SELECT substr(timestamp,1,10) AS day, count(*) AS scans
             FROM scans
             WHERE timestamp > datetime('now','-7 days')
```

```
              GROUP BY day
              ORDER BY day DESC"

# Score distribution by language
wrangler d1 execute preston-check-telemetry --remote \
  --command "SELECT lang, avg(score) AS avg_score, count(*) AS n
             FROM scans GROUP BY lang ORDER BY n DESC"

# Tier distribution
wrangler d1 execute preston-check-telemetry --remote \
  --command "SELECT tier, count(*) FROM scans GROUP BY tier"

# Latest 10 scans
wrangler d1 execute preston-check-telemetry --remote \
  --command "SELECT id, version, tier, lang, score, timestamp
             FROM scans ORDER BY id DESC LIMIT 10"

# Find repos with declining scores (potential churn signal)
wrangler d1 execute preston-check-telemetry --remote \
  --command "SELECT repo_hash,
                    min(score) AS lowest,
                    max(score) AS highest,
                    count(*) AS scans
             FROM scans
             GROUP BY repo_hash
             HAVING max(score) - min(score) > 20
             ORDER BY scans DESC LIMIT 20"
```

# Schema migrations

Schema changes require a migration:

```
# 1. Update workers/telemetry/schema.sql with the new SQL
# 2. Apply remotely
wrangler d1 execute preston-check-telemetry --remote \
  --file=workers/telemetry/migration-2026-05-04.sql

# 3. Update the Worker code to read/write the new column
# 4. Deploy the Worker
```

Cloudflare D1 supports standard SQLite syntax. ALTER TABLE works, but adding a NOT NULL column without a default fails on existing rows — use `ALTER TABLE … ADD COLUMN <name> <type> DEFAULT <value>`.

# Backup

D1 has built-in time-travel restore (point-in-time recovery up to 30 days). For belt-and-suspenders backup:

```
# Export the entire scans table to local JSON
wrangler d1 execute preston-check-telemetry --remote --json \
  --command "SELECT * FROM scans" > backup-$(date +%Y%m%d).json
```

Run weekly via a cron or GitHub Actions schedule once the dataset volume warrants it.

# Privacy and access

D1 access is gated by the same Cloudflare API token that gates the rest of the deploy. Three concentric circles of access:

1. **Operator** — has the API token + the local `wrangler d1 execute` flow. Full read/write.
2. **Worker / Pages Function** — has D1 binding via wrangler.toml. Can INSERT and SELECT but only as part of the running code.
3. **Public** — no direct D1 access. The only way data flows out is via the customer portal's peer-comparison features (which read from KV aggregations, not raw rows) and the annual report.

# Cross-links

- **Telemetry endpoint manual**: `docs/manuals/telemetry-endpoint.md`
- **Standalone Worker manual**: `docs/manuals/standalone-worker.md`
- **Privacy contract**: `docs/telemetry.md`
- **Annual report methodology**: `docs/state-of-fintech-security/2026.md`